

Machine Learning (CAI-262)

Objectives:

- Hierarchical Clustering
 - DBSCAN
-

Hierarchical Clustering

Hierarchical clustering is a connectivity-based clustering model that groups the data points together that are close to each other based on the measure of **similarity or distance**. The assumption is that data points that are close to each other are more similar or related than data points that are farther apart.

A **dendrogram**, a tree-like figure produced by hierarchical clustering, depicts the hierarchical relationships between groups. Individual data points are located at the bottom of the dendrogram, while the largest clusters, which include all the data points, are located at the top. In order to generate different numbers of clusters, the dendrogram can be sliced at various heights.

The dendrogram is created by iteratively merging or splitting clusters based on a measure of similarity or distance between data points. Clusters are divided or merged repeatedly until all data points are contained within a single cluster, or until the predetermined number of clusters is attained. We can look at the dendrogram and measure the height at which the branches of the dendrogram form distinct clusters to calculate the ideal number of clusters. The dendrogram can be sliced at this height to determine the number of clusters.

Types of Hierarchical Clustering

Basically, there are two types of hierarchical Clustering:

1. Agglomerative Clustering
2. Divisive clustering

Hierarchical Agglomerative Clustering

It is also known as the bottom-up approach or hierarchical agglomerative clustering (HAC). A structure that is more informative than the unstructured set of clusters returned by flat clustering. This clustering algorithm does not require us to prespecify the number of clusters. Bottom-up algorithms treat each data as a singleton cluster at the outset and then successively agglomerate pairs of clusters until all clusters have been merged into a single cluster that contains all data.

Hierarchical Divisive clustering

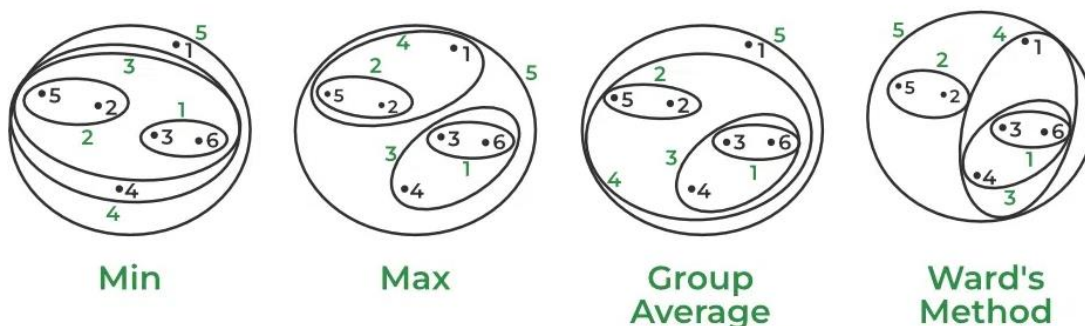
It is also known as a top-down approach. This algorithm also does not require to prespecify the number of clusters. Top-down clustering requires a method for splitting a cluster that contains the whole data and proceeds by splitting clusters recursively until individual data have been split into singleton clusters.

Computing Distance Matrix

While merging two clusters we check the distance between two every pair of clusters and merge the pair with the least distance/most similarity. But the question is how is that distance determined. There are different ways of defining Inter Cluster distance/similarity. Some of them are:

1. **Min Distance:** Find the minimum distance between any two points of the cluster.
2. **Max Distance:** Find the maximum distance between any two points of the cluster.
3. **Group Average:** Find the average distance between every two points of the clusters.
4. **Ward's Method:** The similarity of two clusters is based on the increase in squared error when two clusters are merged.

Distance Matrix Comparison in Hierarchical Clustering



Algorithm of Agglomerative Hierarchical Clustering

-
- 1: Compute the proximity matrix, if necessary.
 - 2: **repeat**
 - 3: Merge the closest two clusters.
 - 4: Update the proximity matrix to reflect the proximity between the new cluster and the original clusters.
 - 5: **until** Only one cluster remains.
-

Example1: Complete Link or MAX version of hierarchical clustering

Clustering starts by computing a distance between every pair of units that you want to cluster. A distance matrix will be symmetric (because the distance between x and y is the same as the distance between y and x) and will have zeroes on the diagonal (because every item is distance zero from itself). The table below is an example of a distance matrix. Only the lower triangle is shown, because the upper triangle can be filled in by reflection.

	1	2	3	4	5
1	0				
2	9	0			
3	3	7	0		
4	6	5	9	0	
5	11	10	2	8	0

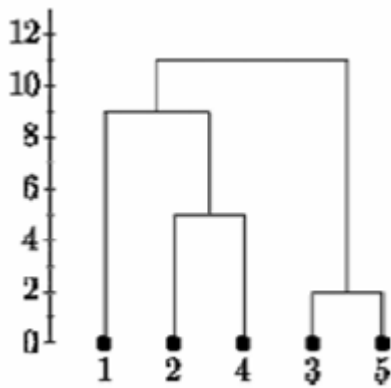
Now let's start clustering. The smallest distance is between 3 and 5 and they get linked up or merged first into a cluster '35'.

To obtain the new distance matrix, we need to remove the 3 and 5 entries, and replace it by an entry "35". Since we are using complete linkage clustering, the distance between "35" and every other item is the maximum of the distance between this item and 3 and this item and 5. For example, $d(1,3)=3$ and $d(1,5)=11$. So, $D(1,"35")=11$. This gives us the new distance matrix. The items with the smallest distance get clustered next. This will be 2 and 4.

	35	1	2	4
35	0			
1	11	0		
2	10	9	0	
4	9	6	5	0

Continuing in this way, after 6 steps, everything is clustered. This is summarized below. On this plot, the y-axis shows the distance between the objects at the time they were clustered. This is called the cluster height. Different visualizations use different measures of cluster height.

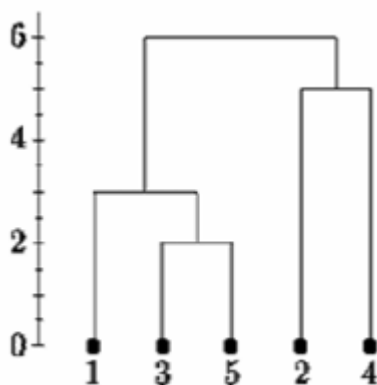
Below is the single linkage dendrogram for the same distance matrix. It starts with cluster "35" but the distance between "35" and each item is now the minimum of $d(x,3)$ and $d(x,5)$. So $c(1,"35")=3$.



Example2: Single link or MIN version of hierarchical clustering

	1	2	3	4	5
1	0				
2	9	0			
3	3	7	0		
4	6	5	9	0	
5	11	10	2	8	0

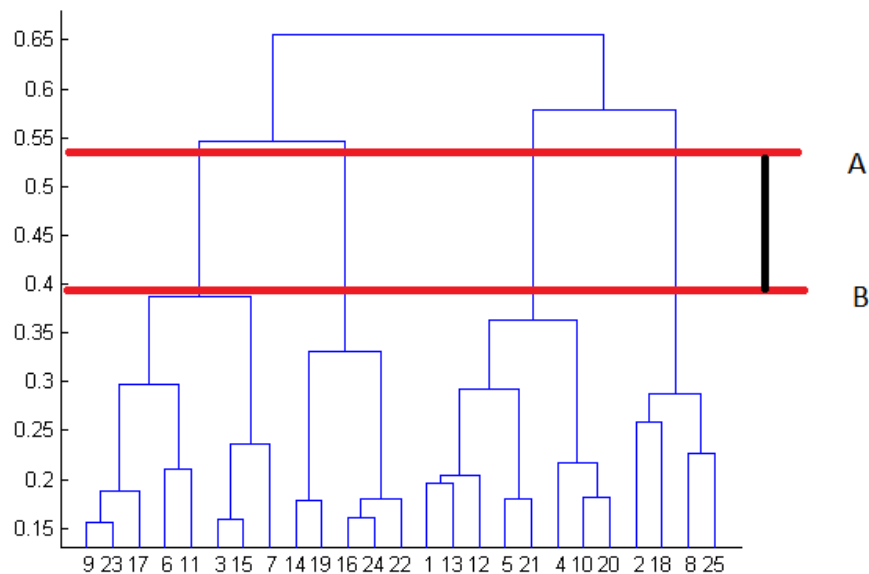
Below is the single linkage dendrogram for the same distance matrix. It starts with cluster "35" but the distance between "35" and each item is now the minimum of $d(x,3)$ and $d(x,5)$. So $c(1, "35")=3$.



Determining Number of clusters

One of the problems with hierarchical clustering is that there is no objective way to say how many clusters there are. To take the decision for the number of clusters that can best depict different clusters can be chosen by carefully observing the dendrogram formed during the algorithm run. The best choice of the number of clusters is the number of vertical lines in the

dendrogram intersect by a horizontal line that can transverse the maximum distance vertically without intersecting a cluster.



In the above example, the best choice of the number of clusters will be 4 since the red horizontal line in the dendrogram below covers the maximum vertical distance AB.

DBSCAN

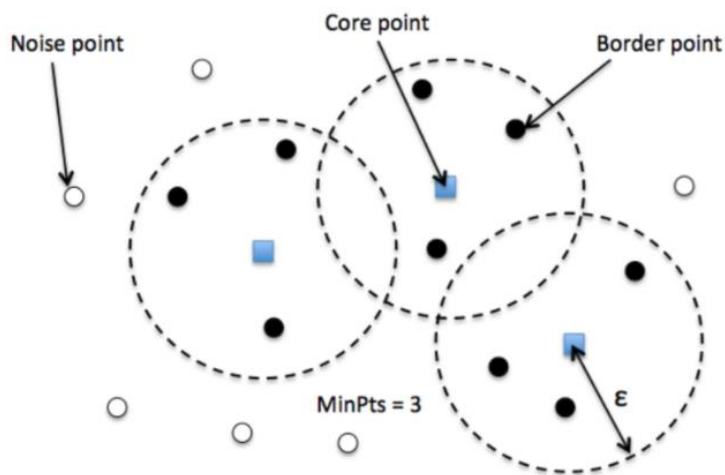
Density-Based Spatial Clustering of Applications with Noise (DBSCAN) is a base algorithm for density-based clustering. It can discover clusters of different shapes and sizes from a large amount of data, which is containing noise and outliers.

The DBSCAN algorithm uses two parameters:

- **minPts:** The minimum number of points (a threshold) clustered together for a region to be considered dense.
- **eps (ϵ):** A distance measure that will be used to locate the points in the neighbourhood of any point.

There are three types of points after the DBSCAN clustering is complete:

- **Core** — This is a point that has at least m points within distance n from itself.
- **Border** — This is a point that has at least one Core point at a distance n .
- **Noise** — This is a point that is neither a Core nor a Border. And it has less than m points within distance n from itself.



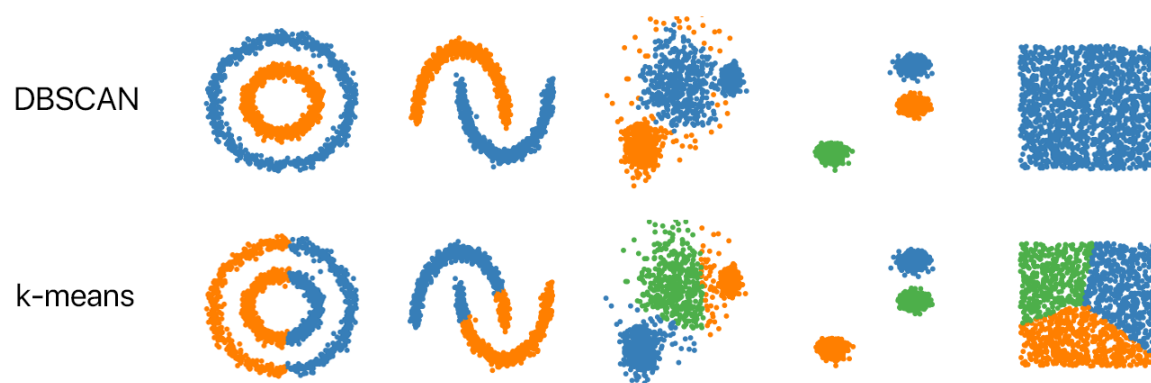
For DBSCAN, the parameters ϵ and **minPts** are needed.

- minPts:** As a rule of thumb, a minimum *minPts* can be derived from the number of dimensions D in the data set, as $\text{minPts} \geq D + 1$. The low value $\text{minPts} = 1$ does not make sense, as then every point on its own will already be a cluster. With $\text{minPts} \leq 2$, the result will be the same as of hierarchical clustering with the single link metric, with the dendrogram cut at height ϵ . Therefore, *minPts* must be chosen at least 3. However, larger values are usually better for data sets with noise and will yield more significant clusters. As a rule of thumb, $\text{minPts} = 2 * D$ can be used, but it may be necessary to choose larger values for very large data, for noisy data or for data that contains many duplicates.
- ϵ :** The value for ϵ can then be chosen by using a k -distance graph, plotting the distance to the $k = \text{minPts} - 1$ nearest neighbour ordered from the largest to the smallest value. Good values of ϵ are where this plot shows an “elbow”: if ϵ is chosen much too small, a large part of the data will not be clustered; whereas for a too high value of ϵ , clusters will merge and the majority of objects will be in the same cluster. In general, small values of ϵ are preferable, and as a rule of thumb, only a small fraction of points should be within this distance of each other.
- Distance function:** The choice of distance function is tightly linked to the choice of ϵ , and has a major impact on the outcomes. In general, it will be necessary to first identify a reasonable measure of similarity for the data set, before the parameter ϵ can be chosen. There is no estimation for this parameter, but the distance functions need to be chosen appropriately for the data set.

Why we need DBSCAN?

K-Means clustering may cluster loosely related observations together. Every observation becomes a part of some cluster eventually, even if the observations are scattered far away in the vector space. Since clusters depend on the mean value of cluster elements, each data point plays a role in forming the clusters. A slight change in data points *might* affect the clustering outcome. This problem is greatly reduced in DBSCAN due to the way clusters are formed. This is usually not a big problem unless we come across some odd shape data.

Another challenge with k -means is that you need to specify the number of clusters (“ k ”) in order to use it. Much of the time, we won’t know what a reasonable k value is *a priori*. What’s nice about DBSCAN is that you don’t have to specify the number of clusters to use it. All you need is a function to calculate the distance between values and some guidance for what amount of distance is considered “close”. DBSCAN also produces more reasonable results than k -means across a variety of different distributions. Below figure illustrates the fact:



Example:

• Apply the DBSCAN algorithm to the given data points and	Data Points:	
• Create the clusters with	P1: (3, 7)	P2: (4, 6)
• minPts = 4 and	P3: (5, 5)	P4: (6, 4)
• epsilon (ϵ) = 1.9.	P5: (7, 3)	P6: (6, 2)
	P7: (7, 2)	P8: (8, 4)
	P9: (3, 3)	P10: (2, 6)
	P11: (3, 5)	P12: (2, 4)

Now calculate distance among all points using Euclidean distance formula

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12
P1	0											
P2	1.41	0										
P3	2.83	1.41	0									
P4	4.24	2.83	1.41	0								
P5	5.66	4.24	2.83	1.41	0							
P6	5.83	4.47	3.16	2.00	1.41	0						
P7	6.40	5.00	3.61	2.24	1.00	1.00	0					
P8	5.83	4.47	3.16	2.00	1.41	2.83	2.24	0				
P9	4.00	3.16	2.83	3.16	4.00	3.16	4.12	5.10	0			
P10	1.41	2.00	3.16	4.47	5.83	5.66	6.40	6.32	3.16	0		
P11	2.00	1.41	2.00	3.16	4.47	4.24	5.00	5.10	2.00	1.41	0	
P12	3.16	2.83	3.16	4.00	5.10	4.47	5.39	6.00	1.41	2.00	1.41	0

Now find out neighbours of each point residing within given distance epsilon, and check out status of each point (Core, Border, Noise/outlier)

Point	Neighbours within distance eps	Status
P1	P2 , mP10	Border
P2	P1 , P3 , P11	Core
P3	P2 , P4	Border
P4	P3 , P5	Border
P5	P4 , P6 , P7 , P8	Core
P6	P5 , P7	Border
P7	P5 , P6	Border
P8	P5	Border
P9	P12	Noise
P10	P1 , P11	Border
P11	P2 , P10 , P12	Core
P12	P9 , P11	Border

Now plot the data and mark clusters on it

